

Instructor Handout 8: I/O Techniques

Interrupts, Polling, Synchronous/Asynchronous Buses, DMA, and IOP

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 11, 2026

Instructor Copy — Not for Student Distribution

8.1 Before You Teach This Lecture

- **Hamacher, *Computer Organization and Embedded Systems*, 6th ed. (the edition on disk) — the week’s anchor, page-verified 2026-08-11.** Two ranges carry the whole lecture:
 - Ch. 3, pp. 95–119 — the programmer’s view of I/O: §3.1 memory-mapped I/O and the device interface (p. 96–97), §3.1.1 interface functions (p. 97), §3.1.2–3.1.4 polling loops KIN/DOUT and the READWAIT/TELEWAIT patterns (p. 97–103), §3.2 interrupts and the five-step scenario (p. 103–104), §3.2.1 enabling/disabling via IE (p. 106–107), §3.2.2 polling-the-sources vs. vectored (p. 107–109), §3.2.4 priority and IPENDING/TELEENABLE (p. 110–111).
 - Ch. 7 §§7.1–7.4, pp. 227–242 — the hardware view: §7.1 bus structure and master/slave roles (p. 228), §7.2 synchronous and asynchronous bus operation (p. 229), §7.2.1 synchronous with wait states (p. 230–233), §7.2.2 the full handshake (p. 233–236), §7.3 arbitration BR/BG (p. 237–238), §7.4 interface circuits (p. 238–239).

Edition warning for your own copy: the text that was indexed in an earlier pass was a different (5th) edition; if you cross-check a page number against a physical copy, confirm the edition first. This 6th-edition file *never uses the terms “DMA” or “IOP”* — §7.3 arbitration is the functional mechanism but the term “DMA” never appears. Phrase that Act honestly (see Topic 4 below).

- **Stallings, *Computer Organization and Architecture*, Ch. 7** — the source for the *names* DMA and IOP. Not page-indexed; cite at chapter level only, never a page number.
- **Handoff from Week 7:** the deck opens by reminding students that Weeks 6–7 built a CPU that controls *itself* (hardwired, then microprogrammed control), and that both designs assumed the CPU is the only thing that matters. The whole of Week 8 is the speed-mismatch consequence of adding the outside world. Run the Week 7 summary frame (or the Week 7 deck’s last slide) if students look lost at the opening.

8.2 Topic 1: Memory-Mapped I/O and Polling

The speed mismatch that motivates everything: a CPU executes roughly one instruction per nanosecond; a keyboard produces roughly one keystroke every hundred million nanoseconds. If the CPU must read every keystroke, it spends that gap doing nothing unless it has a plan. The simplest plan is **memory-mapped I/O**: give each device a small set of registers — a data register, a status register, a control register — that occupy ordinary addresses in the CPU’s address space, so the CPU reads and writes them with ordinary load/store instructions and needs no special I/O instructions. Six functions appear in every device interface: data register, status register, control register, address decoding, timing generation, and format conversion (Hamacher, 6th ed., §3.1/§3.1.1/§7.4, pp. 96–97, 238–239).

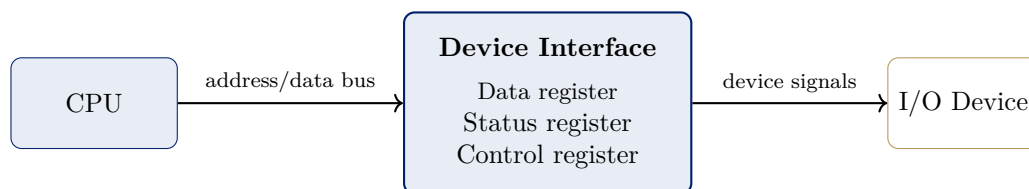


Figure 8.1: The device-interface model — CPU → interface (data/status/control registers) → device. The interface is where the CPU’s bus protocol meets the device’s physical signals.

How to present it. Draw the CPU box first, then the device-interface box with its three registers labelled, then the device box — but *before* drawing the interface, ask “where does the CPU’s world end and the keyboard’s begin?” so students place the seam themselves. Then add the two arrows with their labels (**address/data bus** on the left, **device signals** on the right) and point out that the left arrow is exactly the same kind of wire the CPU uses to talk to memory — that is the entire point of memory-mapped I/O.

Definition. Program-controlled I/O (polling): the CPU repeatedly reads a device’s *status register* in a loop, checking a ready flag, until the device signals ready — then performs the data transfer itself. New notation: KIN (keyboard-input status flag, set when a keystroke is ready in KBD_DATA) and DOUT (display-output status flag, set when the display is ready for the next character in DISP_DATA). The CPU is the only party that ever initiates; the device never does.

Example 8.2.1 (The polling loops, written on the board). **Board:** *write these two three-line loops side by side, then walk the keyboard one instruction at a time:*

READWAIT: Test KIN	WRITEWAIT: Test DOUT
Branch=0 READWAIT	Branch=0 WRITEWAIT
MoveByte KBD_DATA, R	MoveByte R, DISP_DATA

Build order: (1) write both loop bodies first; (2) trace the keyboard loop’s three lines with the two “tick-tock” outcomes (flag still 0 → jump back; flag 1 → fall through); (3) only then point at the display loop and have the class narrate it themselves (“what is different?” — the transfer direction). Each failed pass re-executes *Test/Branch* and does nothing else: the CPU is spinning.

Example 8.2.2 (Quantifying the spin — two instances). **Board:** *work the lecture’s instance first, then a second one live.*

- Lecture instance: 2 GHz CPU, 5 keystrokes/s. $0.2 \text{ s} \times 2 \times 10^9 = 4 \times 10^8$ instructions, at 2 instructions/iteration $\approx 2 \times 10^8$ READWAIT iterations between keystrokes.

- Second instance: *4 GHz CPU, 8 keystrokes/s (0.125 s apart). $0.125 \times 4 \times 10^9 = 5 \times 10^8$ instructions, $\div 2 = 2.5 \times 10^8$ iterations — even more wasted iterations for a faster CPU, because the device’s speed did not change.*

Live-interaction hook: pose the second instance and have students predict whether a faster CPU wastes more or fewer iterations before you reveal the arithmetic. The trap answer (“a faster CPU wastes less” — wrong, the device interval is fixed, so more instructions fit in it) is the whole point of the slide.

Teaching note: The classic student slip here is “polling is bad.” It is not bad — it is *simple and free of hardware*, and for one slow device with nothing else to do it is the right answer (a temperature sensor polled once a second costs nothing). Keep the judgment one axis: *how much of the CPU does watching cost*. That axis is the spine of the whole lecture and returns in the head-to-head table in Topic 4.

8.3 Topic 2: Interrupt-Driven I/O

Polling wastes cycles because the CPU is the only party allowed to initiate. The reversal: let the device signal the CPU when it has something to report. **Interrupt:** a signal from a device that suspends the CPU’s current program, transfers control to a fixed handler — the **interrupt-service routine (ISR)** — and resumes the original program exactly where it left off. Two pieces of notation: the delay between request and ISR start is **interrupt latency**; registers the ISR must not disturb are saved in **shadow registers**.

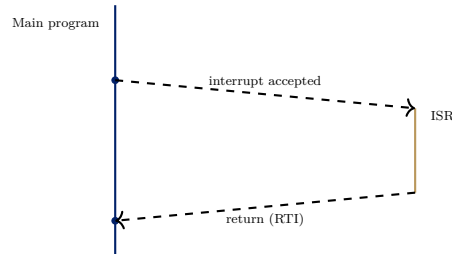


Figure 8.2: Interrupt as a transfer of control: main program suspended at the interrupt point, control jumps to the ISR, and RTI returns exactly where it left off.

Enabling and disabling — two independent switches. A request reaches the CPU only when *both* agree: the per-device enable bit in that device’s control register (KIRQ for the keyboard, DIRQ for the display) *and* the single global IE bit inside the processor status register (PS). IE= 0 masks every request regardless of source — the global panic button; the per-device bits answer “is *this* device even allowed to ask?”.

The five-step handling scenario (Hamacher, 6th ed., §3.2.1, p. 106–107), applied to the keyboard with KIRQ= 1, PS.IE= 1:

1. The keyboard sets KIN= 1 and raises an interrupt-request line.
2. The CPU finishes its *current instruction* (never mid-instruction), then checks PS.IE.
3. Enabled: CPU saves PC (and registers the ISR will clobber) into shadow registers, clears PS.IE← 0, loads PC with the ISR’s starting address.

4. The ISR services the device (`MoveByte KBD_DATA, R`, which also clears `KIN`), then executes `RTI`.
5. `RTI` restores `PC` and sets `PS.IE ← 1`; the interrupted program resumes with no memory of being interrupted.

Example 8.3.1 (The five steps drawn on the board). **Board:** *redraw the transfer-of-control figure as a vertical timeline and annotate each step beside the timeline as you narrate it — build order: (1) the two vertical timelines (main program left, ISR right); (2) the two dashed arrows labelled **interrupt accepted** and **return (RTI)**; (3) a “step 1–5” column down the left edge, filled in as you tell the story. **Live question before step 2:** “the keystroke arrives while the CPU is half-way through an instruction — can it jump now?” (No — it must finish the current instruction; that is why step 2 says finishes its current instruction.)*

Handling multiple devices. One request line says *that* something needs attention, not *which*. Two designs: **polling the sources** (the ISR checks each device’s status flag in a fixed order; simple, but dispatch time grows with device count) and **vectored interrupts** (each device supplies an identifying code; hardware indexes an **interrupt-vector table** — a table of ISR starting addresses, one entry per device — and jumps straight there; constant-time dispatch for a little extra hardware).

Example 8.3.2 (Two devices request simultaneously — a “draw the next state” hook). **Board:** *draw the register-level mechanism (Device 1 IRQ and Device 2 IRQ feeding **IPENDING**, **IENABLE** masking, both converging on the priority + vector-table lookup that outputs the ISR start address to `PC`). Then pose the hook question: “Device 1 (higher priority) and Device 2 both request on the same cycle; Device 1’s ISR is still running when Device 2’s request would normally be granted — what happens to Device 2’s request?” Let a student answer before you reveal: if nesting is enabled and Device 2 outranks Device 1’s current ISR priority, the identical five-step scenario applies recursively (Device 1’s ISR plays the “main program”); if nesting is disabled (or Device 2 is lower priority), the request stays **latched in IPENDING** — not lost, not re-requested — until Device 1’s ISR re-enables interrupts and the encoder re-evaluates.*

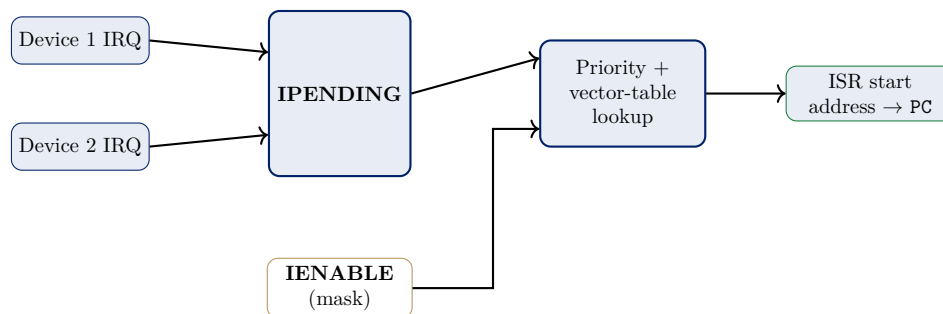


Figure 8.3: Interrupt dispatch with priority: device IRQ lines set bits in **IPENDING**; **IENABLE** masks which pending requests the priority encoder considers; the encoder plus vector-table lookup produces the winning ISR’s starting address.

Teaching note: The single most valuable sentence of this Act: *polling and interrupts are the same problem with the initiative reversed* — they move the waiting cost from CPU cycles to the hardware that detects readiness. Students who hear that once can answer the trade-off table from first principles instead of memorizing it. Also flag explicitly that interrupts do not eliminate the waiting; they relocate it.

8.4 Topic 3: Synchronous and Asynchronous Buses

Drop one level, from “when does the CPU check / get notified” to “what do the wires actually do.” Every interface hangs off a single shared **bus** (address, data, control lines everyone listens to). A **bus protocol** defines how a **master** (the device driving the transfer, usually the CPU) and a **slave** (the responder) coordinate one transfer without colliding. The question: when a master says “take this data,” how does the slave say “not ready yet” or “here it is”? Two answers: **synchronous** and **asynchronous**.

Synchronous bus: all devices share a common **bus clock**; the master places address/data at a fixed clock edge; every slave samples at the *same* edge with no per-transfer negotiation. A slow slave stretches the transfer by holding **Slave-ready** low for extra clock cycles (wait states); the master simply waits, still synchronized.

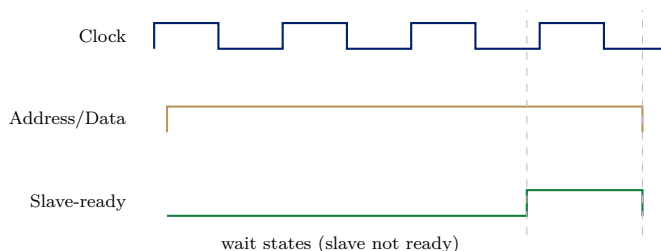


Figure 8.4: Synchronous-bus timing with wait states: address/data held valid across the transfer while **Slave-ready** stays low for several clock periods until the slow slave is ready.

Asynchronous bus (full handshake): no shared clock; master and slave exchange explicit **Master-ready/Slave-ready** signals, each changing only *in response to* the other. Four interlocked steps: (1) **Master-ready** asserted; (2) **Slave-ready** (data valid); (3) **Master-ready** withdrawn (latched); (4) **Slave-ready** withdrawn. Each step waits for the previous one, so **bus skew** (wires arriving at slightly different times) never causes a false sample — but every transfer pays two full round trips.

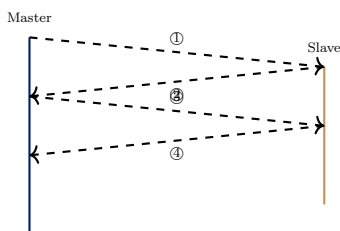


Figure 8.5: The asynchronous full handshake as a four-step interlocked exchange; each arrow fires only after the previous one has been observed.

Example 8.4.1 (Drawing both timing diagrams — two board exercises). **Board (synchronous):** draw the clock row first (a square wave), then the address/data band held valid, then **Slave-ready** as a step rising late. Narrate the two dashed edges: the first dashed line is where **Slave-ready** finally rises; the second is where the master samples. **Live question:** “a fast device needs zero wait states — where would **Slave-ready** sit on this diagram?” (Already high at the very first edge.)

Board (asynchronous): draw the master column and slave column first, then add the four diagonal arrows one at a time, narrating each step’s trigger: “step 3 only fires because the master

saw step 2.” **Prediction trap:** “why does this never sample corrupted data?” Let a student state “because each side waits until it actually observes the other side’s signal” before you add the skew label.

Example 8.4.2 (Wait-state arithmetic — second instance). **Board:** the lecture’s trade-off table says synchronous is cheap per transfer but pays wait states for slow devices. Quantify it: a synchronous transfer needs 3 wait-state cycles at a 250 MHz bus clock ($1/(250 \times 10^6) = 4 \text{ ns}$ per cycle) before *Slave-ready* rises, then one more period to sample: total $(3 + 1) \times 4 = 16 \text{ ns}$. A second instance: 5 wait states at 500 MHz (2 ns/cycle) $\Rightarrow (5 + 1) \times 2 = 12 \text{ ns}$. Do the arithmetic on the board before showing the asynchronous handshake, so the “two round trips” cost lands as a concrete comparison.

Teaching note: Students reliably conclude “asynchronous is strictly better” because it tolerates skew and any speed. The counter is the per-transfer overhead: every asynchronous transfer pays two full round trips, which a fast, similar-speed synchronous bus never pays. That is precisely why high-speed processor buses are synchronous. Have the class *argue* the “why not always async?” Socratic question from the deck — it forces the trade-off table into a sentence.

8.5 Topic 4: DMA and I/O Processors

Interrupts removed the CPU’s *waiting*, but not its involvement in *moving* the data. A disk block transfer is one word at a time; with interrupt-driven I/O the ISR executes on every word (read status, read/write data, update pointer, decrement counter, return). A 4096-byte block at 4 bytes/word is $4096/4 = 1024$ ISR invocations for a single block. The next step removes the CPU from the data path.

Bus arbitration — a third role for a device. DMA needs a capability not yet introduced: a device other than the CPU becoming bus master. **Bus arbitration** decides which of several candidate masters gets the bus each cycle. Each candidate asserts a bus-request line (BR); the arbiter grants the bus to exactly one requester, by priority, over a bus-grant line (BG).

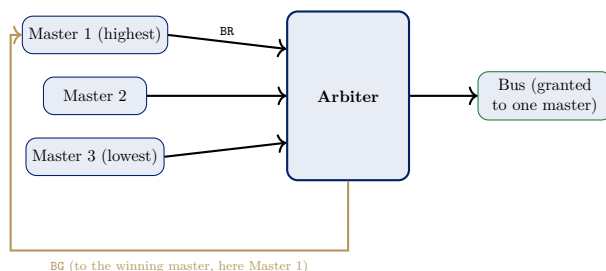


Figure 8.6: Bus arbitration: three candidate masters assert BR; the arbiter grants BG to exactly one requester, by priority, at a time.

Example 8.5.1 (Arbitration order — two instances). **Board:** draw the three masters, the arbiter box, and the BR lines first, then the BG return path, then label the priority order. Narrate the lecture’s instance (all three request the same cycle $\Rightarrow$ grant Master 1; on release, re-evaluate the still-pending requests and grant Master 2 next, then Master 3). **Live question:** “add a Master 4 with lowest priority, all four requesting on the same cycle, each releasing promptly — what is the service order?” (Strict priority: 1, 2, 3, 4 — the queue is served purely by priority because all were pending from the start and no new requests reorder it.)

From arbitration to DMA. Once a device can become bus master, it can transfer data *directly to/from memory*, with no CPU instruction touching the data; the CPU is interrupted once, when the whole block finishes. **Honest-phrasing note:** this 6th edition of Hamacher describes exactly this mechanism in §7.3 but never names it “DMA”; the term and standard treatment follow Stallings Ch. 7 (chapter-level, not page-indexed).

DMA (general/standard treatment — Stallings Ch. 7). A DMA controller is given a source/destination address and a word count by the CPU, becomes bus master (via arbitration), and transfers the whole block directly to/from memory, interrupting the CPU only when the *block* completes. Pro: CPU involvement drops from one ISR per word (1024 for the running 4096-byte block) to one setup + one completion interrupt — constant in block size. Con: the DMA controller and CPU now compete for the same memory bus (**cycle stealing**); extra hardware required.

IOP (general/standard treatment — Stallings Ch. 7). An I/O processor generalizes DMA one step further: a dedicated processor given a whole *channel program* (several transfers, formats, simple decisions) executes it independently, interrupting the main CPU only when the entire program completes. DMA offloads *moving*; an IOP offloads *managing a sequence* — the difference is how much decision-making is delegated away from the CPU.

Example 8.5.2 (ISR count vs. DMA cost — second instance). **Board:** *recompute the lecture’s 4096-byte example for an 8192-byte block (4 bytes/word): interrupt-driven needs $8192/4 = 2048$ ISR invocations; DMA still needs exactly 2 CPU events (1 setup + 1 completion interrupt).* **Prediction trap before revealing:** “guess how many ISR invocations an 8192-byte block needs, given the 4096-byte case needed 1024.” (Exactly double — 2048, linear in block size — while DMA’s 2 is unchanged, which is the entire argument for DMA.) Draw the cycle-stealing sketch while narrating: the DMA controller and CPU both want the same memory bus; a stolen cycle is a CPU memory access that has to wait.

Teaching note: Two misconceptions recur. (1) “DMA halts the CPU.” No — it steals *bus cycles*, so the CPU is delayed only when it wants the memory bus, and only for one cycle at a time, not halted wholesale. (2) “Polling vs. interrupt is a ranking of quality.” It is a *latency-vs-overhead* trade-off that depends on the device’s rate and how much else the CPU has to do. The closing head-to-head table (CPU cost per word and per block for Polling/Interrupts/DMA/IOP) is the artifact students should be able to reconstruct, not memorize — every cell follows from “who is on the critical path.”

8.6 Summary

The one-sentence spine: every I/O technique moves the cost of “someone must notice the device is ready, and someone must move the data” somewhere else — CPU cycles (polling), interrupt-handling hardware (interrupts), per-transfer wire delay (asynchronous buses), or dedicated data-path hardware (DMA/IOP) — and none of them is free.

- **Memory-mapped I/O** places device registers in the normal address space; **polling** tests a status flag (KIN/DOU) in a loop and wastes CPU time in proportion to the device/CPU speed gap.
- **Interrupts** reverse the initiative: PS.IE and per-device enables (KIRQ/DIRQ) gate delivery; **vectored interrupts** dispatch in constant time via IPENDING/IENABLE and the interrupt-vector table; nesting lets a higher-priority request interrupt an ISR recursively, otherwise requests stay latched in IPENDING.

- **Synchronous buses** trust a shared clock (cheap per transfer, wait states for slow slaves); **asynchronous buses** trust nothing (a **Master-ready/Slave-ready** full handshake — skew-immune, but two round trips per transfer).
- **Arbitration** (BR/BG) lets a device become bus master — the basis for **DMA** (block transfer, one completion interrupt, cycle stealing) and **IOP** (whole channel programs; general treatment per Stallings Ch. 7).

Bridge to Week 9: Weeks 1–8 built a CPU that computes, controls itself, and now moves data to and from slow devices. Week 9 asks the same speed-mismatch question about *memory itself* (DRAM is far slower than the CPU): cache memory, mapping functions, and replacement policies.